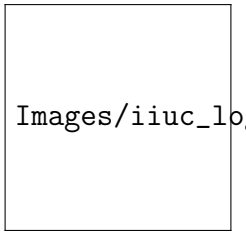


INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

CacheFlow: Memory-Aware Expert Scheduling for Efficient Mixture-of-Experts Models

Supervised by:

Jamil As Ad

Lecturer

Department of Computer Science and Engineering

International Islamic University Chittagong

Submitted By:

Mir Md. Tarhimul Quader	C221017
Md. Iftaker Ahamed Sayem	C221020
Turja Dutta	C221026

BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

Department of Computer Science and Engineering

International Islamic University Chittagong

February 2026

Declaration

We hereby affirm the following statements regarding our thesis:

1. The thesis has been successfully completed as part of our undergraduate degree program at International Islamic University Chittagong.
2. The thesis work does not contain any previously published or third-party content without proper citation.
3. The thesis work has not been previously submitted for any other degree or diploma at any other university or institution.
4. We have appropriately acknowledged all significant sources of contribution in the thesis.

Student's Full Name and Metric ID:

Mir Md. Tarhimul Quader (C221017)

Md. Iftaker Ahamed Sayem (C221020)

Turja Dutta (C221026)

Supervisor's Declaration

I formally state that I have examined this thesis and claim it to be of sufficient quality and scope to be granted the undergraduate degree of Bachelor of Science in Computer Science and Engineering.

.....

Jamil As-Ad
Lecturer
Department of Computer Science and Engineering
International Islamic University Chittagong

Dedication

This thesis report is dedicated to us, our supervisor, and our family. The teamwork was satisfactory and the family's support was incredibly Amazing.

Our dedicated and most hard-working Supervisor and co-supervisor, who has been a constant support throughout these months. In this document, the contributions are acknowledged too.

Acknowledgment

To start with, All the praises to the Almighty Allah, for his mercy because of which we were able to finish our thesis despite having so many obstacles.

Secondly, we would like to extend our gratitude to our supervisor, **Jamil As Ad** Sir, for his continuous effort and guidelines from the very beginning of our research.

Ethical Statement

Hereby we state that None of the unethical practices were used in the completion of our thesis work. The data we used for the research purpose are original. We carefully checked every citation we used here. The writers of the work accept all the liabilities for any kind of violation of the thesis rule.

Abstract

Mixture-of-Experts (MoE) architectures provide the ability to provide large language models at reasonable scale, in that only a small number of expert networks are executed per-token. This is though sparseness reduces compute costs, countering inference time deployment since most systems retain all experts in a memory on a single, high-performance, device, which demand high memory bandwidth and cause out-of-memory errors on a standard computer. This thesis presents the priority based scheduler CacheFlow that renders MoE inference memory-efficient. CacheFlow is a Python middleware, which is connected to the forward pass of a rebuilt Qwen2-MoE model, which provides a direct control of who the experts execute and which ones remain in memory. It sees the GPU memory as a small expert cache and employs the lightweight priority function that takes into account the prevalence of context reuse, proximity of experts. Experiments using real inference and quantized weights demonstrate that consistent results may be achieved even using a small number of working experts. By reducing in-memory expert count, ease of 60 experts down to 16, CacheFlow reduces the active expert memory by approximately 73% and reduces the total amounts of gpu memory utilization by 47-50%, including the static parts of the model, which remain in memory. Consequently, even large MoE models of about 14 billion parameters can be trained on hardware where standard inference algorithms do not perform. Other works show that CacheFlow retains equitable expert delivery, holds high temporal locality in expert utilization, and is significantly memory efficient, employing about 7,132.5 MB less with the use of GPUs in inference. These results demonstrate that the use of memory-conscious scheduling is a successful method of resolving the theoretical aspects of scaling of MoE into practice and practical memory-intensive deployments.

Keywords: CacheFlow, Expert Scheduling, GPU Memory Management, Large Language Models, Memory-Constrained Inference, Mixture-of-Experts, MoE Inference.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Research Background

Large language models (LLMs) have been the cornerstone of contemporary artificial intelligence systems in the last ten years. They drive conversational agents, code generation as well as automated reasoning. Since the performance of models increases with the number of parameters, scientists are trying to design models that grow in capacity without corresponding increase in cost of computation. Mixture-of-Experts (MoE) architectures are up to the task. MoE models can support counts of parameters that are dramatically larger but the computation per token is still manageable by simply routing each token to a small set of specialized expert networks.

Although the MoE models are efficient, they introduce new inference-time issues, namely, in terms of memory consumption. Although the number of experts called is only a few per token, their majority of inference systems assume experts remain active in the GPU memory. In the case of modern MoE models where dozens of experts are involved this results in massive GPU memory requirements that surpass consumer-scale hardware. Therefore, to implement state of the art MoE models, special servers with huge VRAM tend to be used and only a limited number of researchers and developers can access it.

System-level developments have primarily aimed at fashioning attention computation and key-value (KV) cache controls reduction of latency and throughput enhancement during inference. Although the techniques help increase performance when used with dense transformers, it does not address the fundamental problem of core memory congestion of expert-intensive architectures. Expert parameter memory can easily take up most of the memory in MoE systems, and the memory can be squandered where large numbers of experts are needed infrequently.

Due to this lack, the interest in dynamic expert management that characterizes experts as modular components and not permanently resident parameters turns out to increase. Through loading, storing, or evicting experts dynamically according to run time demands, memory usage can be consistent with are run time behavior. The systems to do so are not easily designed: naive eviction may interfere with the implementation of tokens, corrupted attention state, or be extremely slow with an expert changing hands frequently.

The *CacheFlow* solution to these problems is a software middleware to support memory conscious MoE inference with priority based expert scheduling. Rather than dropping all experts each execution *CacheFlow* at the MoE layer interrupts and stores a fixed number of experts into GPU memory. The framework ensures correct inference and minimizes the pressure on memory greatly by taking key-value context reuse and expert residency into account in its scheduling. This is part of a broader trend of considering big neural models as dynamically-controlled systems, on which taking care with scheduling and memory management is crucial to making any realistic use.

1.2 Problem Statement

The objective of MoE architectures is to enhance computational efficiency to the level of activating few expert networks on an input token. Theoretically, this sparsity allows very large models to be run efficiently when making inferences. Practically, however, most frameworks of MoE inference store all the experts in GPU memory throughout the run. That selection is incongruent to the thin-computing and thick-memory footprint.

The growth of MoE models logarithmically depends on the number of expert parameters required to encourage overall consumption of the GPUs. In consumer GPUs with small VRAM, it may invoke out -of-memory errors or forceful quantization and model pruning which negatively affect accuracy. Fixed expert placement of a small number of experts at a time ensures that the unused experts are available in memory, leading to scaling and wastage.

The current countermeasures are typically simple caching or eviction techniques such as LRU, or load experts in an on demand and synchronous fashion. They ignore fundamental MoE inference characteristics: maintaining key-value context across tokens, the varying payer of smartly using versus loading a smart, and obtainability of pattern of dynamical access contributed by token routing. Therefore, inconsequential

policies make experts waste time on thrashing and stalling pipelines.

This is the main issue which is solved in this thesis and thus it can be stated as follows:

> *How effectively can Mixture-of-Experts inference be implemented on hardware with resource limitations (such as available memory): How does the controller efficiently schedule expert execution through dynamically managing expert residency on the temporal memory of a GPU and retain its correctness, reduce its latency, and not excessively swap experts between experts?*

The resolution of this problem requires an execution- and memory-conscious scheduler. It has to make real-time decisions about which experts have to remain in GPU memory, which have to be evicted and what requests are to be served also at aabiding by a limited memory budget. Absence of such a solution is harmful to the implementation of large MoE models, and triggers the development of *CacheFlow*, a priority based dynamic expert scheduling model.

1.3 Motivation of the Research

The fast increase in Mixture-of-Experts (MoE) language model has demonstrated that sparse activation can be used to achieve disproportionate growth in model capacity without corresponding increasing cost. However, field implementation has not yet been used extensively due to the capacity by inference time memory factors. In practice, in most real-world applications, researchers are using consumer-level GPUs with limited VRAM, in which loading big MoE models at the start of program execution leaves them unusable. This disconnect between the possibilities of architecture and the reality of deployment is the main driving force behind this study.

The previous optimization aspects in large language model inference have mostly centred on enhancing computational performance e.g. accelerating attention or minimizing key-value (KV) cache overhead. Although these improvements result in large performance because of the gigantic performance advantages of MoE models, they are not related to the most effective source of memory utilization in MoE models: expert parameters. Consequently, it can still happen that systems that can be effectively used to process dense models fail when applied to expert-heavy architectures, when only a small fraction of the experts are actually interested in execution.

The other driving force is the poor management of experts by the existing methods. Making all experts permanently resident parameters dismisses the dynamism and load responsive use of experts. Unfortunately, pattern of access by expert users can be very non-uniform and determined by prompt content, sequence length and routing behavior. The inference systems are left without exploiting this locality, and therefore incur avoidable wastage in memory resources and unnecessary performance degradation through cache thrashing and the use of stalled execution.

Another reason this research is being conducted is the need to democratize large-scale MoE models. Between the stages of instantiating state-of-the-art language model on constrained hardware and on more general systems, enabling efficient inference can reduce the energy cost of experimentation, education and deployment so that smaller research groups and organizations can use the latest language models, without investing in special hardware. In terms of systems, this goes with a more general trend of looking at neural models as dynamically administered workloads in which memory is actively coordinated but not an absolute resource allocation.

Such a gap is why *CacheFlow* is driven by the desire to provide a practical solution, on a software level, to fill this gap. The framework aims to match what it needs to match the usage of its memory or inference demand using priority based expert scheduling and restricted expert residency. The ensuing system proves that with respect to proper scheduling and memory-conscious execution, they can achieve a significant savings in terms of GPU memory consumption and simultaneously maintain correctness and consistent performance, allowing larger MoE models to be more accessible and usable in resource-restrained worsted scenarios.

1.4 Objectives

The main purpose of the study is to develop and test a memory-aware scheduling model, which will allow the effective Mixture of Experts (MoE) inference on the GPUs with limited memory. In order to achieve this objective, the study produces a useful software middleware that cater disciplinely oversees which experts are retained within the GPU memory and ensures inference the right path and performance is constant.

The specific objectives of this research are as follows:

- To develop a priority based skilled vacation procedure that determines a subset of the experts to stay in the GPU memory subject to a given capacity limit.

- To develop a working Python middleware to inference MoE which interrupts at the MoE layer and processes expert loading and eviction at real inference with quantized model weights.
- To minimize the expert memory footprint in the dynamic external memory through maintaining a small expert working set in the GPU memory, but still be able to correctly run large MoE models.
- To examine the connection between inference behavior and cache capacity such as the latency stability, cache thrashing and expert reuse behavior.
- To test the performance of the workload items in terms of prompt locality and reuse of key values on the cache hit rate and system performance.
- To establish the generality of the proposed framework by demonstrating that the architecture can be generalized to other MoE families by packaging their blocks of expertise.

Collectively, these goals guide the design and implementation of *CacheFlow*, ensuring the framework addresses both the theoretical and practical challenges of deploying large-scale MoE models in memory-constrained environments.

1.5 Organization of the Thesis

The thesis contains five chapters, and each of them is focused on one of the aspects of the research problem and its proposed solution.

Chapter 1 provides the background on the topic of the research by describing Mixture-of-Experts (MoE) architectures and the difficulty of executing inference with scarce memory. It indicates the issue, motivation, and goals and the framework of the thesis.

Chapter 2 is a literature review on large language models, systems of MoE, and inference time memory management. It revisits the available methods of expert routing, caching, and scheduling, and highlights the weaknesses that the new framework will resolve.

Chapter 3 shows the methodology of the research. It outlines how *CacheFlow* was designed such as priority-based scheduling, expert resident control, and the workflow. The parameters of the experimental environment, integration of models, and

the specifics of implementing middleware are also found in the chapter.

Chapter 4 is on experimental results and discussion. It evaluates the ability of *CacheFlow* in minimizing the memory usage, latency and performance of the cache. The analysis points to constant working-set trends, those conditions, which cause thrashing and the impact of timely locality on performance.

In **chapter 5**, the thesis closes by summarizing the main contributions and commenting on the implications of the results, as well as specifying the possible directions of the research in the future.

1.6 Summary

This chapter provided a background of the research by following the history of Mixture-of-Experts (MoE) architectures and finding the challenges on their implementation throughout the inference process on the memory-limited hardware. MoE models are sparsely activated in order to save computation, but have limited practical use. Poor memory utilization and limitations of scalability caused by the common belief that all experts need to be kept in the GPU memory.

The problem as outlined in the thesis stated that an expert-management system was required to be memory-aware and able to perform efficient MoE inference but at the same time stay correct and stable. The reason was that the theoretical advantages of MoE models were contrasted with their practical deployability, namely on consumer-grade GPUs. To overcome this gap, the proposed objectives of the research were established, and the design and analysis of a priority-based expert-scheduling framework were focused upon.

Finally, the thesis organization was outlined. The following chapter presents a comprehensive literature review, covering Large Language Models, MoE system architectures, and existing inference-time memory management techniques, providing the theoretical foundation for the proposed *CacheFlow* framework.

Chapter 2

Literature Review

2.1 Overview

The modern large-scale models of language have developed based on the breakthroughs in terms of architecture, optimization, and systems design within the last three decades or so. Initial literature on Mixture-of-Experts (MoE) models proposed the use of conditional computation: a subsection of the model can specialize on specific areas of inputs. This was done with the goal of increasing the representational power of the network without corresponding increases in the cost of computation. Simultaneously, the development of optimization algorithms, normalization algorithms, and attention systems made the deep neural networks effective in scaling. These advances led to the development of Transformer-based architectures which were soon the standard design of sequence modeling. The more complex the models became in terms of millions of parameters, then billions of parameters, then even of trillions of parameters, the greater the increase in performance became in direct proportion to the corresponding rise in both computation and memory requirements.

Transformer changed the face of natural language processing. It rendered training very parallelisable and offered effective long -range dependency modeling. This was later followed up by more research that enhanced expressiveness, stability and scalability. Further innovations included as better normalisation techniques, new activation functions, sparse attention systems, and large scale distributed training systems. Although these advances increased the quality of models and the speed of training, they increased the level of inference to resource requirements. In dense ones, all of the parameters are required to be held in memory at inference, posing a difficulty in deploying the technology to a low-hardware device.

The response to the scaling problem was based on the sparse Mixture-of-Experts. MoE models incurred a massive number of parameters without proportionate raise in per-token compute by simply enabling a limited number of experts per-token.

However, as we have seen, inference systems do require all expert weights to be loaded on the GPU. As a result, the number of memory used tends to increase with the overall number of the experts, rather than the number of the experts at work. This restricts the capability of consumer GPUs to execute large MoE models.

Recent studies have changed the focus on inference time optimization and improved memory management. Efficient attention, key-value cache memory paging, task switching between the CPU and the GPU, and distributed sharding are some of the techniques that are expected to minimize memory bottlenecks. Although useful, these approaches are usually operated at a coarse granularity, not scheduling context of a specific system, and are oriented towards multi-gpu implementation instead of single-gpu environment. Therefore, there is still a research void of fine-grained, context-dependent expert scheduling on one single-processor graphics card.

In this chapter, a review of essential MoE literature, Transformer design, scaling techniques, and current inference systems takes the structure of a paper-by-paper. Both works are analyzed in terms of their idea, technical contribution, implementation, and empirical findings, as well as limitations, primarily in terms of inference time memory use. The history of the review: Mixture-of-Experts designs, the emergence of Transformers, large-scale sparse models, and current serving and memory-management designs. The chapter also concludes with a comparative summary and a definite research gap that drives CacheFlow.

2.2 Foundational and Modern Architectures for Scalable Neural Models

This part presents systematic, paper-by-paper discussions of traditional Mixture-of-Experts, approach to optimization, Transformer structures, large scale language model, and current inferential computations. The content of each subsection is a balanced paragraph, where one of the works presents its important idea, method, results, and limitations, especially concerning the inference-time memory scalability.

2.2.1 Adaptive Mixtures of Local Experts by Jacobs et al. (1991)

Jacobs et al. (1991) portrayed Adaptive Mixtures of Local Experts. Instead, they constructed a neural network that had several expert networks, each specializing in a segment of the input space. The process of a gating network, which is dynamically trying to assign weight to the output of each expert, is based on the soft partitioning process of probabilistic nature. An approximation of functionality in experiments was

higher than approximate converging monolithic networks. Nonetheless, the work concentrated on small-scale systems and it did not take into account memory limitations of hardware. The entire expert parameters were assumed to be loaded on inference, which upon the scale was a reasonable assumption but became an issue.

2.2.2 Hierarchical Mixtures of Experts by Jordan and Jacobs (1994)

Jordan and Jacobs (1994) built on the Mixture-of-Experts framework to introduce hierarchical gating structures which were optimized using Expectation-Maximisation. Davids would now have the ability to specialise at various levels, which would result in a higher degree of flexibility and capacity. Compared to flat mixtures the hierarchical design was better at classification and regression. As in the previous work, it also assumed all the experts were in memory at runtime and it did not concern runtime memory efficiency. However, the hierarchical concept also forms the basis of the contemporary sparse Transformer-based designs, according to which experts are conditionally activated, but usually reside in memory.

2.2.3 Adam: A Method for Stochastic Optimization by Kingma and Ba (2014)

Kingma and Ba (2014) presented Adam, which is an optimizer that combines adaptive learning rates and momentum to accelerate training. Adam follows first and second-order moments of gradients, stabilizing updates in large-scale training. It was made a default option of deep Transformers and MoE models due to its strength and little tuning. Adam on the other hand is only able to touch on the training but has no influence on the inference-time usage of memory. It assisted scaling training, but failed to alleviate the parameter-resident bottleneck that has arisen in implementing those trained models.

2.2.4 Distilling the Knowledge in a Neural Network by Hinton et al. (2015)

The authors Hinton et al. (2015) presented a method of knowledge distillation which is a more compact student model that has the accuracy of a larger neural network. The process takes the softened output probabilities of the teacher and transfers it to the student causing a number of parameters to be reduced in order to attain competitive performance. It has been experimentally verified that distilled models can be as accurate as larger networks by re-training, but rather than managing memory

dynamically during inference with a large model (which requires substantial memory), the compressed model gives a fixed map due to their minimal size. Therefore, distillation can only help in reducing memory use at the model level; it is less flexible with regards to expert-level memory management that is needed by sparse architectures.

2.2.5 Neural Machine Translation by Jointly Learning to Align and Translate by Bahdanau et al. (2015)

To simplify neural machine translation, Bahdanau et al. (2015) introduced attention to the decoding process to enable a model to pay attention to relevant input tokens. Their recurrent networks calculate weights on alignment at each decoding step and this enhances the quality of translation and long sequences are better handled. Attention however, raises the usage of intermediate memory due to the usage of alignment matrices and stored hidden states. Although this inspired Transformer, it does not cause the size of the parameter memory to be smaller; all the model weights are kept loaded to inference.

2.2.6 Batch Normalization by Ioffe and Szegedy (2015)

Ioffe and Szegedy (2015) suggested Batch normalization which stabilizes and accelerates the training process through normalization of layer inputs across mini-batches. It causes a lower covariate shift inside the model and permits the greater learning rates resulting in quicker convergence and higher generalization. However, Batch Normalization is essentially training friendly; it does not reduce the number of parameters and memory imprint of the model. In perception, the acquired scaling and shifting parameters remain functional, and the process does not foster the scalability of memory. Its primary contribution is performance with regard to trainings as opposed to the resource management on runtime mode.

2.2.7 Layer Normalization by Ba et al. (2016)

Two-dimensional normalization was introduced by Ba et al. (2016) as Layer Normalization that normalizes features activations, rather than batch ones. The method works better when batches statistics are volatile in recurrent and sequence models. Transformer architectures now incorporate Layer Normalization as a part of them. It increases training stability and convergence, and does not lower the demands of inference run time memory; during execution all the model parameters are resident in the GPU memory. It, therefore, adds architectural stabilization, rather than memory optimization.

2.2.8 Gaussian Error Linear Units (GELU) by Hendrycks and Gimpel (2016)

Hendrycks and Gimpel (2016) proposed a combination of ReLU and stochastic regularization, and called it the GELU activation function. GELU enhances the performance on the language and vision in Transformer feed-forward layers. Even while it has a positive impact on representational power, it does not alter the model size or memory footprint. Activation functions affect Churchill but not the need to load all weight matrices when making inferences. As such, its influence is on performance, rather than on the use of memory.

2.2.9 Attention Is All You Need by Vaswani et al. (2017)

The Transformer suggested by Vaswani et al. (2017) substitutes recurrent and convolutional sequence models with multi-headed self-attention and position-wise feed-forward networks. The architecture supports paralleling input token processing which enables high level of training efficiencies and scalability. Translate Experiments of machine translation demonstrate state-of-the-art by performance and lower training time than recurrent networks. Transformers however put all the parameters in memory at inference and therefore with increase in the model size, the memory requirements of the GPU also increase significantly. Although repetition bottlenecks are removed in the architecture, inference time memory scaling, which became important in large scale deployments was not addressed.

2.2.10 Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer by Shazeer et al. (2017)

Shazeer et al. (2017) proposed sparsely-gated Mixture-of-Experts layers, with the capacity to add parameter capacity but no expansion of per-token computation. Gating network chooses a small selection of experts per input whereas the experts allow conditional computation. It is shown that there is substantial improvement in language modeling and translation with computation manageable by experiments. However, not many experts are activated on a token, although the majority of systems maintain all expert parameters in memory, as part of inference. Thus, the use of memory depends on the number of total experts and not active ones. This separation of computational sparsity and memory density is one of the central points of later studies, such as CacheFlow.

2.2.11 Efficient Softmax Approximation for GPUs by Grave et al. (2017)

Grave et al. (2017) suggested effective softmax approximations in order to accelerate large-vocabulary language modeling on GPUs. They calculate the hierarchical clustering and adaptive partitioning method to minimize the computational complexity in the output layers. The training and inference time in experiments is faster than in standard softmax. Nevertheless, although the method is less costly in arithmetic, it does not reduce the requirement commonly required by all model parameters being resident at inference time. The focus of optimization is output-layer computation instead of the memory footprint and therefore is not concerned with the issue of weight residency in large Transformer or MoE models.

2.2.12 Improving Language Understanding by Generative Pre-Training by Radford et al. (2018)

The article by Radford et al. (2018) presented a new method of NLP in generative pre-training followed by fine-tuning as a potent tool. They use large language models based on transformers, which they trained on large unlabeled corpora, and distilled to downstream with minimal learning. The tests revealed significant gains in such benchmarking as question answering and textual entailment. The designs are based on dense Transformer layers, where all the parameters are opened during inference. The model supported the tendency of increasing the performance with the number of parameters, although it was smaller than subsequent large systems. It greatly developed transfer learning in NLP but failed to take into account memory limitations when running at runtime and how to manage the parameters dynamically.

2.2.13 Breaking the Softmax Bottleneck by Yang et al. (2018)

Yang et al. (2018) discovered the so-called softmax bottleneck, the constraint of expressiveness of typical softmax layers in language models. They put forward a high rank factorization that upscales representational capacity but at a minimal computational cost. On language-modeling benchmarks, it was found that there was a decreased perplexity. The approach, though, does not decrease the number of stored parameters or modify assumptions about memory residency: all of the learned matrices have to be available in order to perform inference. In turn, inference-time memory pressure, particularly that of large Transformer models, does not decrease, as modeling capacity is improved.

2.2.14 Language Models are Unsupervised Multitask Learners by Radford et al. (2019)

.

GPT-2 was presented by Radford et al. (2019), who showed that the zero-shot setting allows large-scale language models which have been trained on a variety of web data to handle a number of tasks. The model takes a deep Transformer decoder that has much more parameters than previous systems. The findings indicate high performance in translation, summary and question answering without fine-tuning. Yet it is a completely dense architecture, and must have full parameter residency on inference. The size of the model (billions of parameters) renders the use of GPUs memory intensive, which tends to increase deployment costs due to memory limits.

2.2.15 Generating Long Sequences with Sparse Transformers by Child et al. (2019)

To reduce the quadratic cost of self-attention, Child et al. (2019) presented Sparse Transformers. They encode orderly sparsity in attention matrix that allows the sequential processing of long sequences at a lower rate of memory and computation. Better results in experiments on long context language models are demonstrated. This low density reimbursement attention-related memory artifact, but not residency of model weights. In inference, all the feedforward and projection parameters are loaded in their entirety. Therefore, attention sparsity helps in alleviating sequence-length scaling, but does not tackle the bigger problem of expert weight memory in large MoE systems.

2.2.16 Root Mean Square Layer Normalization by Zhang and Sennrich (2019)

Zhang and Sennrich (2019) suggested a simplified version of Layer Normalization called Root Mean Square Layer Normalization (RMSNorm). As computational efficiency and numerical stability RMSNorm is a variant of mean-centered variance-normalized normalization. Tests have demonstrated performance similar and better than standard Layer Normalization in Transformers. RMSNorm is a computationally less costly but does not modify the overall number of the parameters and the memory residency of the parameters in the inference. It makes its contribution in optimization and stability rather than scalability of the memory usage. Even large Transformer and MoE models based on RMSNorm require full weight residency.

2.2.17 Language Models are Few-Shot Learners by Brown et al. (2020)

GPT architecture Transformers were expanded to 175 billion parameters, presented by Brown et al. (2020). The primary contribution is that the demonstration of powerful few-shot and zero-shot learning without either fine-tuning or scaled down to huge sizes. The dense decoder is trained using huge datasets using distributed infrastructure. Findings depict significant improvements on NLP benchmarks (question answering and translation). The dense structure however requires full parameter residency to be applied in inference, and therefore cannot be deployed locally with specialized clusters. Although GPT -3 confirms scaling laws with respect to performance, it illustrates drastic limitations of dense large-scale models in memory.

2.2.18 Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer by Raffel et al. (2020)

Raffel et al. (2020) suggested T5, which resolves all NLP tasks to a single text-to-text model. The structure is built based on a standard Transformer encoder-decoder that is trained on a huge corpus that is under curation. It shows high transfer-learning on various benchmarks. Although T5 enhances training methodology and task unification, the subnet has a dense execution of parameters that necessitate an entire model reservoir throughout the inference phase. When T5 is scaled up, the amount of memory required by the GPU is scaled up accordingly. The work makes progress on modeling generality but no progress on inference time memory constraints.

2.2.19 DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters by Rasley et al. (2020)

Rasley et al. (2020), proposed system-level DeepSpeed, which is an efficient training of extremely large models by optimization of memory and distributed partitioning. The ZeRO (Zero Redundancy Optimizer) of DeepSpeed divides optimizer states and gradients in the devices which allows less memory duplication through training. Experiments demonstrate that it is possible to train models of more than 100 billion parameters on commodity GPU clusters. DeepSpeed is however mainly aimed at training efficiency and distributed scaling, and not optimization of single node inference. Dense models even during inference still need full parameter residency, unless they are specifically partitioned. DeepSpeed cuts, therefore, reduce but not eliminate, the redundancy of memory accessible to of-the-shelf inference on a single gpu.

2.2.20 An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale by Dosovitskiy et al. (2020)

Dosovitskiy et al., (2020) proposed the Vision Transformer (ViT) that uses the Transformer architecture to deal with image recognition by splitting an image into patches that can be regarded as tokens. The model is able to equal convolutional neural networks on massive datasets, interface attention-based designs outside of NLP. Similarly to language Transformers, ViTs use dense parameter network models which also require full memory residency during inference. The larger model depth and width the more memory memory of a graphics card also require. Although the work extends the use of Transformer, it does not change the memory scaling properties.

2.2.21 GLU Variants Improve Transformer by Shazeer (2020)

Gated Linear Unit (GLU) variants were presented by Shazeer (2020) as variants of Transformer feed-forward layers focused on increasing their expressiveness. These variants introduce gating mechanisms to linear transformations and hence the variants are able to better skilled modeling capacity and enhance training stability. Experiments indicate that GLU versions do better with a variety of language-modeling tasks than do the normal feed-forward layers. The changes also, on some parameters, increase computational load and parameter count. Notably, there is no net reduction in inference-time memory GLU does not require the use of memory to decrease inference-time Memories: all transformed feed-forward parameters have to remain in memory during model execution, which is a dense-memory assumption.

2.2.22 Longformer: The Long-Document Transformer by Beltagy et al. (2020)

Proposed by Beltagy et al. (2020), the Longformer is the Transformer dispensing the entire idea of self-attention with a combination of local windowed attention and sparse global attention. The method reduces the quadratic memory and calculate cost of attention on long documents. Longformer has also been experimentally found to scale to long-sequence tasks, such as document classification and question answering, much better. Longformer is more economical in terms of attention-buffer memory, but the fact remains that not only do all model weights remain loaded when making inferences, but they are also held there. Attributes that represent the attention activations still take a backseat compared to the parameters in terms of occupying the GPU memory footprint.

2.2.23 ALBERT: A Lite BERT for Self-Supervised Learning of Language Representations by Lan et al. (2020)

Lan et al. (2020) proposed ALBERT, a downsized version of BERT, which downsizes model size without degrading performance through sharing the weights across the layers and factorizing the embedding matrices. These transformations decrease significantly the number of parameters total as compared to BERT. It has been experimentally demonstrated that ALBERT obtains competitive accuracy with less memory consumption. Nevertheless, it is the architectural compression, rather than the dynamic run time control, that is remembered. ALBERT reduces the number of parameters which are always static but does not have adaptive memory scheduling or conditional expert activation. As such it is a compression method as opposed to a runtime inference optimizer.

2.2.24 GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding by Lepikhin et al. (2021)

Lepikhin et al. (2021) introduced GShard, which is a system that scales massive models with conditional computation as well as automatic sharding of parameters introduced in distributed hardware. It uses sparse layers of Mixture-of-Experts on Transformers and separates experts among modified devices so as to balance memory and computing duties. Experiments indicate that GShard has the ability to train very large MoE models on distributed clusters of TPUs. But the method is directed towards multi-device situations that already possess distributed memory. It makes no effort inference-time memory constraints on one GPU. Special weights remain full in the partition distributed and soft runtime scheduling at hard memory constraints is not investigated.

2.2.25 Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity by Fedus et al. (2022)

Fedus et al. (2022) proposed the Switch Transformer, that is, Mixture-of-experts routing is simplified through only one expert per token (top-1 routing) rather than using many experts. This minimizes the routing overhead and enhances stability of the training, enabling models to scale to trillion parameters. Experimental optimization has shown that Switch Transformers are equal to dense models in terms of cost per token. Nevertheless, the inference systems usually maintain all expert parameters

in memory to support dynamic routing, and therefore the memory consumption is directly proportional to the number of experts as opposed to the quantity of experts that are actually active. In this way, even though Switch Transformers can promote sparse scaling, they have no implication of automatically addressing inference-time memory residency issues.

2.2.26 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness by Dao et al. (2022)

FlashAttention is an algorithm demonstrated by Dao et al. (2022) as it reduces memory overhead and accelerates self-attention. Focusing on memory-access patterns and being IO-aware, FlashAttention makes very precise attention calculations and at a far smaller memory footprint. Standard experiments demonstrate an increased throughput and low-memory consumption associated with processing long sequences in contrast to the usual attention. However, FlashAttention is concerned with the activation memory rather than the model parameters that are static. If it does not lower the need to have all the model weights such as expert weights in memory. Thus, FlashAttention can ease scale-to-sequence but is not a more powerful way of overcoming the expert-weight bottleneck in memory.

2.2.27 FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning by Dao (2023)

FlashAttention was further improved to a more-parallel organization and workload partitioning by Dao (2023) to create FlashAttention 2. It actively serving device is increasing the usage of the graphics card, reducing synchronization costs, which translates to additional performance and memory-saving improvements over the base. Similarly to FlashAttention, FlashAttention-2 does not focus on the parameter storage of Transformers or MoE models but instead, intermediate activation memory and compute efficiency. It therefore enhances the performance of the process of attention execution but fails to address the memory residency at the expertise level among sparse architectures.

2.2.28 Efficient Memory Management for Large Language Model Serving with PagedAttention by Kwon et al. (2023)

Kwon et al. (2023) proposed the memory-based attention method, which is PagedAttention, which can improve the efficiency of large-language-model serving. The

technique virtualizes key-value (KV) cache storage using paging of memory instead of consecutive blocks that make it be less prone to fragmentation and better utilize the GPU. Experiments demonstrate that memory waste and throughput are less in the case of long-context inference. Yet, PagedAttention only deals with the memory in terms of states of attention, as opposed to the model parameters. It is not a solution to maintaining all model weights, and, in particular, expert weights in MoE models, in GPU memory. In this way, the technique is a more efficient way of serving, but does not eliminate expert-level memory bottlenecks.

2.2.29 LLaMA: Open and Efficient Foundation Language Models by Touvron et al. (2023)

Touvron et al. (2023) presented a family of open and relatively efficient dense Transformer-based models, the use of which uses optimized strategies of data curation to propose LLaMA. The researchers demonstrated that intelligently developed training pipelines could be used to perform equally well with respect to larger systems using smaller models. Results of the experiment were good on various NLP benchmarks. Nevertheless, LLaMA is a large model that loads all the parameters in memory when performing inference. It also scales its memory requirement with the size of models, limiting its use to consumer GPUs. Thereby, although LLaMA is more efficient in terms of parameter optimization compared to the previous dense models, it lacks the capability to handle dynamic memory management.

2.2.30 SwapMoE: Serving Off-the-Shelf MoE-based Large Language Models with Tunable Memory Budget by Yoo and Lee (2023)

SwapMoE, suggested by Yoo and Lee (2023), is a system tapping expert weights in both CPU and GPU to allow the inference of large MoE models and limited memory. It has a memory limit fixed by the user and automatically transfers experts at on-demand. It was found that experiments indicated lower peak GPU memory and inference with stricter VRAM limits. Swapping is based upon heuristic/LRU-based policy and neither exploits fine-grained context nor any key-value reuse data. The round-trip transfer of CPU to and from GPUs may introduce a latency and stability may be compromised under intermittent loads. SwapMoE deals somewhat with expert memory residency and does not provide very advanced scheduling to reduce thrashing.

2.2.31 MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache by Zuo et al. (2023)

The article by Zuo et al. (2023) proposes MoE-Infinity that leverages expert caching based on the idea of sparsity to execute MoE inferences on personal computers. The system evicts and loads experts depending on patterns of access, reducing the requirements of the GPU memories. Tests demonstrated that MoE models in large scale could be executed on commodity equipment which had managed memory consumption. The defaulting cache is based on basic eviction brush strokes, and does not utilize any token level context or key-value reuse indications. There could be communication overhead in the event of frequent repeated transfers between memory tiers. MoE-Infinity builds upon memory-aware MoE inference, but does not give the scheduling logic the continuity as a forward path of the model.

2.2.32 DeepSpeed-MII: Model Inference Interface by Microsoft (2023)

DeepSpeed-MII offers an inference interface that runs on DeepSpeed distributed optimization infrastructure, that allows serving large-model applications simultaneously using multiple GPUs. The system is based on model parallelism and pipeline using optimized inference to enhance throughput. It is not aimed at fine-grained expert-limited node level scheduling of MoE architectures, but at deploying clusters on a large scale fulfilling single-node memory constraints. Dense parameter residency is the default. DeepSpeed-MII advances the inference scale in a multi-device design, but is not concerned with the expertise weight memory management of restricted GPUs.

2.2.33 TensorRT-LLM: Optimized Inference for Large Language Models by NVIDIA (2023)

TensorRT-LLM is a highly optimized inference engine bringing the use of kernel fusion, optimization of precision, and hardware acceleration to improve the performance of large language models. It minimizes the latency and maximizes throughput by the way of low-level optimization and efficient utilization of GPUs. Choose any model However, TensorRT-LLM supposes that all model weights are resident in AV RAM prior to execution. It does not have active expert scheduling or weight-switching of MoE architectures. Although it accelerates the computation process, memory footprint is still associated with size of total parameters. In this way, TensorRT-LLM

is maximizing speed, without addressing the main memory scale issue of large systems using MoE.

2.2.34 DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model by Liang et al. (2024)

Liang et al. (2024) came up with DeepSeek-V2, a massive Mixture-of-Experts language model that is more efficient in the use of parameters but equally does not compromise its performance. The architecture amalgamates distant expert layers to the Transformer backbone and concentrates on economical scaling in order to reduce the computational cost. Competitive accuracy was demonstrated to be better than dense models of approximately similar effective capacity and greater compute efficiency per token. But inference typically stores all expert weights in which to run dynamic routing are stored in the GPU memory. Therefore, DeepSeek-V2, however, makes scaling efficient, but fails to address the problem of memory residency on limited hardware.

2.2.35 fMoE: Fine-Grained and Prompt-Aware Expert Offloading for Large Mixture-of-Experts Serving by Shen et al. (2024)

Shen et al. (2024) developed fine-grained, prompt-sensitive expert offloading fMoE, an expert offloading mechanism to help to serve the needs of large Mixture-of-Experts model. The system is used to predict expert execution and purges inactive experts to identify prompt features, minimizing the peak memory in the GPU. It was experimentally demonstrated that memory efficiency was enhanced and fewer swaps were occurring compared to the case with the use of a static offloading. fMoE targets distributed serving environments and balances performance across devices. Even though it implements prompt-awareness, it does not schedule on a single GPU such that it is a strict part of the forward path of the model. Coordination overhead and routing analysis can be imposed by high concurrency on latency; fMoE is an expert aware offloading technology that does not consider context offloading with strong single node memory constraints.

2.3 Comparative Summary of Reviewed Systems

Mixture-of-Experts, dense Transformers, scaling innovations, and inference-serving systems are included in the literature. This is despite the fact that these studies enhanced modeling, efficiency and scalability but their approaches to its management

during the inference process vary widely. Very limited amount of systems are concerned with the maintenance of expert weights on the GPU in the event of limited memory. The following table gives the sparsity, expert granularity, memory strategy, deployment focus and inference-time memory constraints of each system.

Analytical Interpretation

The most important patterns in the analysis:

- Dense Architectures and Static Residency:

  Most Transformer-based models store all the parameters in the GPU memory during inference. The weights remain resident, although they can be made cheaper to train, or they can be made much smaller during activations, preventing scalability.

- Sparse Compute vs. Sparse Memory:

  The models of MoE minimize the calculation by utilizing a small number of experts on each token. However, in most instances, all the experts remain loaded even when not in use hence, memory sparsity is nonexistent.

- Offloading-Based Solutions:

  SwapMoE and MoE-Infinity reduce peak VRAM memory by swapping of both the GPU and CPU memory. Such schemes are based on heuristics or LRU eviction, and may bring in transfer delays.

- Distributed-Centric Designs:

  GShard and DeepSpeed are concerned with the optimization of multi-device partitions, as opposed to single-GPU. They also assist in groups but not in a single constrained node.

- Limited Context-Aware Scheduling:

  Very little systems take advantage of token-level context or key-value reuse to make decisions on which experts to retain. The Majority make use of static rules or generic caches.

On the whole, the community came a long way in terms of the computational power and scaling, yet there is still nothing about finer/grained/context-aware work-scheduling on a single-GPU deployment.

Table 2.1: Comparative Summary of Reviewed Architectures and Systems

Related Work	**Sparse Compute**	**MoE Layer**	**Expert Scheduling**	**CPU-GPU Swap**	**Consumer GPU Ready**	**Memory Limitation**
Adaptive MoE (1991)	Yes	Yes	No	No	N/A	No runtime memory management considered.
Transformer (2017)	No	No	No	No	Yes	Full-weight residency required at inference.
Sparsely-Gated MoE (2017)	Yes	Yes	No	No	No	Experts typically remain fully resident.
Switch Transformer (2022)	Yes	Yes	No	No	No	Sparse compute; memory dominated by expert weights.
FlashAttention (2022)	No	No	No	No	Yes	Optimizes attention activations, not weight residency.
PagedAttention (2023)	No	No	No	No	Yes	KV-cache paging only; expert weights unchanged.
SwapMoE (2023)	Yes	Yes	Partial	Yes	Partial	Transfer overhead and thrashing under frequent swaps.
MoE-Infinity (2023)	Yes	Yes	Partial	Yes	Partial	Heuristic caching; limited context prioritization.
DeepSpeed-MII (2023)	No	No	No	Partial	No	Distributed focus; not single-GPU expert scheduling.
TensorRT-LLM (2023)	No	No	No	No	No	Assumes weights already resident in GPU memory.
DeepSeek-V2 (2024)	Yes	Yes	No	No	No	Does not inherently solve constrained expert residency.
fMoE (2024)	Yes	Yes	Partial	Yes	Partial	Designed for multi-device serving; coordination overhead.

2.4 Research Gap

Architecture (above), efficiency, and scaling The above literature review demonstrates advancements in architecture, performance, and size. Conditional computation was introduced by Mixture-of-Experts in order to increase model capacity without proportionately increasing compute. Parallel sequence modeling would then be made possible by transformers, and very large models such as GPT-3 had proved these scaling laws. Recent research addresses sparse routing, sharding, memory-efficient attention, and expert offloading to reduce compute and memory bottlenecks.

Nevertheless, a number of gaps are left open to inference of the limited GPU memory:

- Dense models necessitate the weights taking up all the weight, which is not feasible on consumer-grade devices.
- MoE continues to maintain all the experts loaded and thus memory is a function of the number of total experts and not the active experts.
- The offloading systems are coarse-grained, heuristic driven systems; they might provoke the frequent transfers of data, leading to latency and thrashing, and are oriented to the distributed systems.
- Attention optimizations do not just decrease the memory of the activations, but they primarily do not decrease the storage of the expert weights.

The literature taken together indicate a discrepancy: memory sparsity is not accompanied with arithmetic sparsity. Only a small number of systems take into account fine grained and context sensitive scheduling during the forward pass on one GPU with hard constraints. Expert weights are so far treated by few frameworks and ranked in order by reuse patterns that they can maintain their execution without architecture modification. Such a gap is the inspiration of memory aware scheduling of MoE inference in CacheFlow.

2.5 Summary

Chapter 2 provided a summary of scalable neural models, starting with ancient Mixture-of-Experts to contemporary inference serving models. Mixture-of-Experts This paper presented conditional computation; Transformers presented parallel sequence modeling, scaling tools such as sparse routing and sharding had increased

compute efficiency, but could do nothing about creating inference-time memory constraints. Subsequent literature covered compression, parameter sharing, efficient softmax and activation optimizations. Newer systems introduced memory-aware algorithms, such as KV-cache paging, CPU-GPU swapping, partitioning, although most of them still assume complete expert residency.

Computational sparsity in sparse MoE is not associated with memory sparsity, limiting the memory of GPUs. Those systems are primarily based on dense model acceleration, scaling distribution, or heuristically conducted offloading. Very limited support fine-grained, context-driven scheduling into a single-gpu execution path. This continuous gap leads to the requirement of CacheFlow, which is a framework that dynamically supports expert presence, according to run-time requirement and ensures proper execution.

The second chapter is based on this premise by stating the proposed system architecture and temporal scheduling approach of CacheFlow that would resolve the mentioned limitations of inference-time memory management of Mixture-of-Experts models.

Chapter 3

Methodology

3.1 System Overview

CacheFlow is an architecture that is a hybrid with memory-aware Mixture-of-Experts (MoE) inference to decouple expert scheduling and model execution. It is implemented as Python middleware that is in-built in a forward pass of a hand-written MoE transformer. This enables fine-grained control of the expert residency and execution and satisfies close criteria of the limits between the GPUs and the memory. Figure 3.1 shows the general execution flow and the division between scheduling logic and model execution.

On the high level, *CacheFlow* divides the inference pipeline into two portions, the Scheduler Logic and the Model Execution Engine. The scheduler is considered to be a central decision-maker, as it decides which experts to be loaded into GPU memory any one time. Only the experts then present are forward computed by the engine running the program. This partition provides the opportunity to modify or prolong expert-management policies, but does not require any modification to the basic model.

In the process of inference, input text is run through the tokenizer and generates a sequence of tokens. Considering the movement of the tokens across transformer layers, they strike an MoE block, where a gating network directs them to experts. This is the point when *CacheFlow* will connect with the MoE block. Rather than executing all routed experts at the same time, it puts requests to the scheduler that it executes them with a hard expert-capacity limit.

Each request is assigned a priority score by the scheduler in accordance with the contemporary surroundings as well as cache locality. Expert requests that are already physically co-located in memory or are attached to an existing key -value (KV)

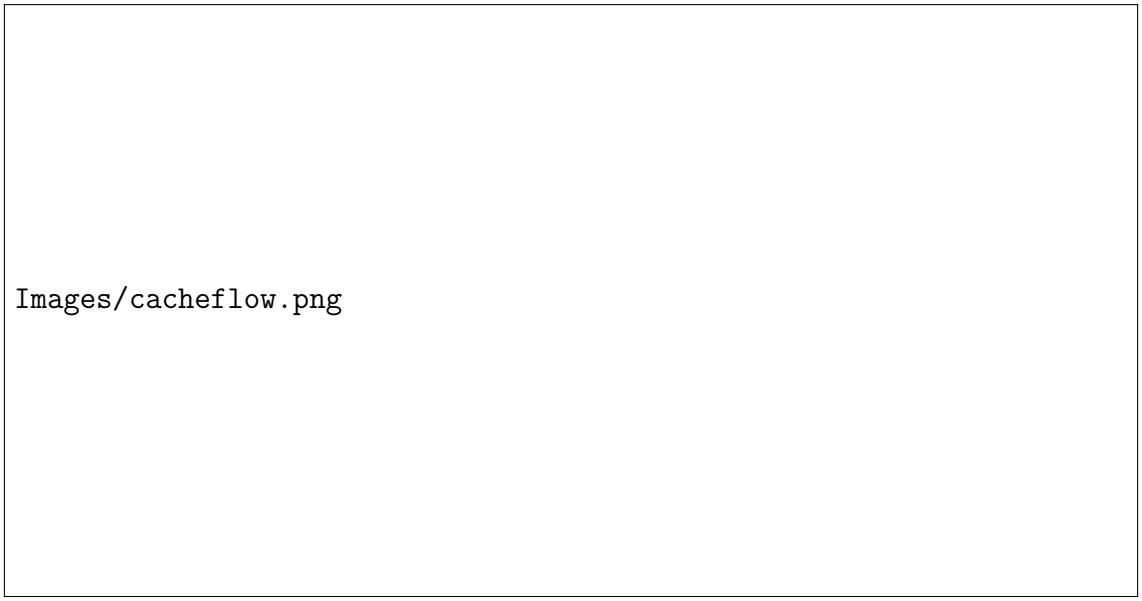
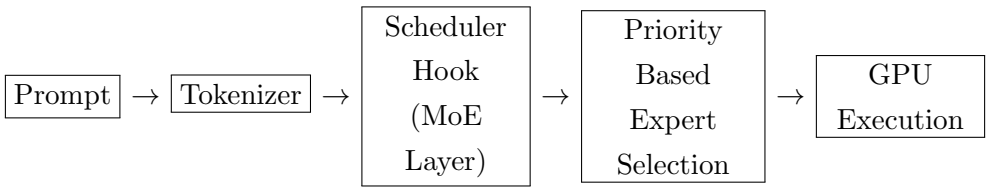


Figure 3.1: CacheFlow system architecture.

scenario have a priority to maintain temporal locality and eliminate unnecessary evictions. The scheduler can be used to determine whether an expert can immediately execute or we have to vacate a currently running resident expert based on these priorities as well as the condition of the current cache.

After the scheduler has been completed, the execution engine executes the chosen experts on the GPU and back the output of these experts through the output of the transformer layer, which is weighted. GPU memory can be in limited number of experts such that only a set number of experts are present at a time, with the number being defined by the expert-capacity parameter. Basic building blocks of embedding model, attention models and shared model parameters remain permanently resident, whereas *CacheFlow* dynamically updates expert parameters.

In a summarized perspective it would look like:



Built-in to the manually reconfigured MoE architecture to execute the scheduling logic, *CacheFlow* ensures that expert-residency is decided within the point of actual execution. The design can enable unmodified models to execute on smaller hardware with accuracy, and has a significantly small memory footprint of large MoE models.

3.2 The CacheFlow Scheduling Algorithm

The main idea of the *CacheFlow* design consists of a priority-based Expert scheduling algorithm which dynamically allocates proactive residency of experts in the GPU memory within a limited capacity constraint. This scheduler is a crucial part of the class named `PyCacheFlowScheduler` which is a centralized controller that ranks expert execution requests, keeps track of resident experts and decides which experts should be evicted in case of memory constraints.

3.2.1 Priority Scoring Mechanism

Every expert execution request sent to the scheduler is scored in terms of its priority which is a measure of the surrounding execution as well as cache locality. The score is computed as:

$$\text{Score} = (W_{kv} \cdot \mathbb{I}_{context}) + (W_{loc} \cdot \mathbb{I}_{resident})$$

Where:

- $\mathbb{I}_{context}$ is an indicator function that takes the value of 1 when the request is related to an existing key value (KV) context, and 0 when the context does not exist.
- $\mathbb{I}_{resident}$ is an indicator function that values 1 when the requested expert is already resident in intellectual memory, and 0 when not.
- W_{kv} and W_{loc} the fixed priority weights which have values 1000 and 500 respectively in the existing implementation.

Such formulation ensures that expert requests that maintain the context of attention or re-use resident experts are given priorities relative to cold requests. The discrete indicator functions maintain the scoring basis of the logic that is simply deterministic and can reduce the scheduling burden in inference.

3.2.2 Request Queue and Ordering

Passing expert request of the form is sent into a request queue. This queue is ordered by the scheduler according to descending order of priority score, then according to arrival order to support fairness between the requests which have equal priority. This

design does not cause starvation and still gives preference to locality preserving executions. After being ordered, requests are batch processed. In a request, the scheduler determines if the requested expert is allowed to be executed on the spot or an eviction must be performed to allow admission into the resident set.

3.2.3 Virtual LRU and Expert Residency Tracking

CacheFlow also utilizes Least Recently Used (LRU) mechanism which is a virtual version and used to trace expert residency. Instead of keeping trails of raw expert parameters, or best on a per-reference basis, of tensors, the scheduler merely keeps an ordered list of integer expert identifiers of the resident set in place at a point in time. The corresponding abstraction enables any decision to be made regarding the residence that does not depend on the exact model weights to minimize the bookkeeping overhead and make eviction logic easier. When an expert is accessed:

- In the event that the expert is already resident, its identifier is shifted to the top of the LRU list indicating it to be most recently used.
- In case the expert is not resident, it is placed on the first spot of the list. When the amount of the insertion is more than the set capacity C, the tail of the list expert identifier is chosen to evict.

This eviction policy guarantees that experts that are less frequently used are evicted earlier, whilst the priority system continues to impact on the requests that are served to be performed initially before eviction takes place.

3.2.4 Eviction Policy and Capacity Constraint

The scheduler imposes a firm expert capacity C and is the maximum population of experts that can currently occupy the GPU memory. In determining cases where a new expert needs admission to the cache, eviction is carried out deterministically according to the virtual LRU order. In-flight executions are not interrupted by eviction decisions but are subject to further scheduling decisions, keeping the executions correct.

A *CacheFlow* scheduling algorithm is a manner to balance it by using priority based requests ordering and a virtual LRU residency model to achieve locality, fairness, and memory efficiency. This architecture enables the system to have a constant working pool of experts with limited memory budgets without the system suffering excessive expert switching and execution stalls.

3.3 Model Implementation (Custom Qwen2-MoE)

The Qwen2-MoE architecture was re-implemented in Python as a new implementation rather than to an already implemented model wrapper to support fine-grained expert scheduling and memory control. The need behind this decision is that, high level implementations to date conceal expert execution through opaque forward passes, without allowing the injection of custom scheduling logic into the analysis at the desired granularity. When applying all of the above components, *CacheFlow* has a direct control over expert routing, execution order and residency management during inference. Figure 3.2 depicts the reconstruction of Qwen2-MoE architecture and its modular parts.

3.3.1 Manual Reconstruction of the MoE Architecture

The reconstructed model is based on the main concepts of Qwen2-MoE, including transformer attention layers, sparse layers of expert-driven feedforward blocks, and top-k routing. These components, attention, gating and expert execution, are done as individual classes. This design is a modular nature that allows *CacheFlow* to capture the flow of execution literally at the MoE layer where expert selection and execution occur.

Shared projections, attention layers, and token embeddings are static and just implemented one time special values that remain in the GPU memory during inference. Expert parameters on the other hand are regarded as dynamically managed objects whose location is managed by scheduled *CacheFlow*.

3.3.2 Sparse Expert Layer: Qwen2MoeMLP

The sparse feedforward model is found in the `Qwen2MoeMLP` class that is a representation of the Mixture-of-Experts block. It hosts a pool of autonomous networks of specialists, which are all lightweight multilayer perceptron (MLPs). As opposed to dense feedforward layers, only a few of these experts are used to activate on each token.

A gating network evaluates the routing score on every token and chooses the top-k experts by the scores. This routing process generates a group of expert identifiers and gating weights which are used to form expert outputs. More importantly, such routing information is not hidden but revealed explicit, which leads to downstream scheduling decisions in advance of expert execution.

Figure 3.2: Qwen2-MoE transformer block.

3.3.3 Gating Network and Top-K Routing

All the experts available are graded in the gating mechanism by each token. The top-K experts are chosen from this distribution, usually with $k \ll N_{experts}$. Such a sparse pattern of activation is essential to MoE efficiency but induces dynamic, highly non-uniform patterns of expert access.

Using the gating logic purely, *CacheFlow* will be able to monitor expert demand executions in-flight and place expert execute requests to the scheduler without any actual computation being performed. This transparency cannot be seen in typical wrappers of the model in which the choice and execution of the experts are very closely bound.

3.3.4 CacheFlow Integration via `CacheFlowQwenBlock`

The `CacheFlowQwenBlock` is the point of integration between the model and *CacheFlow*. This element takes the place of the normal MoE path and is an interception layer in the forward pass of the model. The `CacheFlowQwenBlock` has the following subsequent

processes when a token is received in the MoE block:

1. Gets the best-k expert methods and gating weight of routing network.
2. Forwards knowledgeable execution requests to the `PyCacheFlowScheduler`, such as contextual information such as key-value (KV) reuse.
3. Is provided with scheduling decisions that clarify the clinicians who are to perform according to the existing residency and capacity limitations.
4. Roads only the chosen selective resident professionals and sums up their weight outputs.

This construction will make the expert execution completely controlled by the scheduler still maintaining the mathematical accuracy of the MoE calculation. Expert eviction and admission decisions are made transparent between forward pass, and do not change the functioning of the model.

3.3.5 Rationale for Manual Implementation

Restructuring the Qwen2-MoE architecture is a perceived methodological decision that allows the contribution of *CacheFlow*. This eliminates black-box execution, enabling the system to have capabilities to allow expert-level behavior, and allow inference to make scheduling and memory management a matter of first-class concern. Though the experiments are directly on Qwen2-MoE, the model is model-agnostic and can be generalized to other MoE structures by re-implementing the sparse blocks of these architectures with similar interception points.

3.4 Optimization and Constraints

CacheFlow has been designed to satisfy practical constraints on the size of memory in a GPU, and the need to execute large Mixture-of-Experts models on consumer-grade GPUs. The following section describes the major optimization and constraints that form the system, such as quantization of fixed weights and explicit bounds of the number of experts that can remain in memory.

3.4.1 Memory Constraints and Design Assumptions

In modern MoE model, there are two kinds of parameters, which are, static and expert weights. Token embeddings, attention layers, layer-norm parameters, and shared projections are known as static tasks that, for each of the forward pass, need loading hence that must be permanently reside in the GPU memory. Instead, expert weights

become activated only on selected batches and, thus, they are the primary cause of memory variability during inference.

CacheFlow makes a positive assumption of the maximum accessible GPU memory and regards expert parameters as dynamically consumed resource. The scheduler applies a hard capacity constraint on the number of experts that may be resident in use at any given time keeping the system safely within the memory constraints and avoiding out of memory errors.

3.4.2 4-bit Quantization of Static Weights

CacheFlow loads constants weights with 4-bit quantization to reduce the amount of reserve memory that the memory layout occupies. It utilizes the `BitsAndBytesConfig` set to `NormalFloat4 (NF4)` that reduces the memory consumption substantially without losses in the accuracy. This is exactly the same type of uniform quantization used on embeddings, attention layers, and hermits, allowing professional parameters to use VRAM.

This is also necessary to operating *CacheFlow* on limited hardware, as this will ensure that constant model components will take up a fixed, small quantity of VRAM. The left over memory can later be utilized to load or evict expert parameters on the fly.

3.4.3 Expert Capacity and Residency Limit

To explicitly define this parameter, *CacheFlow* adds a parameter, C, indicating the number of experts that can exist in the memory of a GPU at a given time. This parameter is an abstract VRAM budget of expert weights. The system is able to emulate the various memory constraints by modifying C and examine the influence of expert residency on inference.

In experiments, C is small working sets to the entire pool of expert. The general setup is C=16, where only some portion of the entire pool of experts can remain in memory. This causes the eviction of experts by the scheduler and there are trade offs in terms of cache locality, latency and memory usage.

3.4.4 Execution Correctness Under Constraints

With violent memory constraints and dynamic expert management, *CacheFlow* manages to maintain accurate inference anytime the experts are expelled between execu-

tion phases. Every orchestrated professional action is completed prior to the occurrence of any change in residency. In this way the operational behaviour of the model does not change yet the use of memory is low.

Cacheflow achieves a controlled environment by configuring a expert residency with a limit in terms of capacity plus a monthly fee of static-weight quantization by using large MoE models to execute reliably with a limited memory budget. The analysis below is based on these optimizations.

3.5 Experimental Environment

The *CacheFlow* experimental assessment was done under a controlled setting that represents memory-constrained deployment scenarios under memory-limitative Mixture-of-Experts inference. This section explains the hardware setup, software stack, and workload design used to test the framework's effectiveness.

3.5.1 Hardware Configuration

Every experiment was performed on one NVIDIA T4 with approximately 15GB of available VRAM. T4 It is a common, low-priced accelerator device used in cloud and research labs. The fact that it runs on a single GPU implies that the results are made up of local memory-management decision, and not distributed-system effects.

3.5.2 Software Stack

CacheFlow is developed in Python on PyTorch. Developing model components PyTorch uses the module system, which provides extraordinarily fine-level control over how the execution flow and the other components are located. Its implementation is also compatible with Hugging Face tokenizers and configuration and does not use high-level wrappers that conceal expert implementation.

The `bitsandbytes` library including NF4 4-bit quantization is used to load the quantized models which reduces the memory footprint of the static model. The Accelerate library deals with executing consistency and location of devices. Every run of inference is performed under the evaluation mode where gradients are turned off.

3.5.3 Workload and Prompt Design

Artificial load prompts were to be developed to stress CacheFlow at varying access modes. We used two primary types:

1. Coherent topic prompts emphasize only one domain (e.g. physics questions) to induce temporal locality in the use by experts.

2. Random-topic prompts include irrelevant queries (most topics) which violate locality and emphasize the scheduler.

Such workloads provide a control over systematic research of cache hit rates, expert residency and latency to various routing dynamic conditions. A comparison of the results between scenarios indicates that prompt locality has an impact on the performance of *CacheFlow* in terms of scheduling.

3.5.4 Experimental Protocol

We changed the expert capacity parameter C holding constant the rest of the model and hardware parameters. We measured parameters of latency, expert residence patterns, cache hits, and memory usage, respectively, to each setting. Full expert usage was yet again provoking out-of-memory failures over all target hardware which confirmed itself in the requirement of *CacheFlow*.

Such experimental setup gives a good foundation to the behavior study and performance of *CacheFlow*. The results are presented and discussed later chapters.

3.6 Summary

In this chapter, it was described how the *CacheFlow* framework was implemented with respect to its system design, scheduling algorithm, model implementation and experimental setup. *CacheFlow* is a Python middleware, which is a memory aware Mixture-of-Experts inference. It functions through the separation of expert scheduling decisions and model execution as well as a fixed expert residency limit.

The chapter described the priority-based scheduling algorithm whereby the expert requests are handled. It also has a scoring system, virtual LRU residency tracking and deterministic eviction policy. These elements make sure that professional demands are processed in a way that is efficient and predictable.

In order to have finer control over the execution of the experts, the Qwen2-MoE architecture was re-created by hand. This enabled to simply insert the logic of scheduling into the forward pass of the model. In this way, developers got full visibility of expert routing and execution, which over black-box wrappers, is not possible.

The most important optimization methods were also discussed. These consist of 4-bit quantization of fixed weights and literally limited expert capacity. The two strategies are essential in the operation within limited-GPU memory capabilities.

Lastly, this chapter described the experiment setting. It detailed the hardware setup, software stack configuration and workload configuration to evaluate *CacheFlow* to access patterns.

A combination of these methodological elements is a viable and controllable study of memory conscious expert scheduling. The results and discussion of the experiment will be provided in the following chapter where the performance, behavior and the memory efficiency of *CacheFlow* will be evaluated on actual inference workloads.

Chapter 4

Results and Discussions

4.1 Performance Analysis: The Capacity Trade-off

One of the major objectives of this assessment is to research the effect of the expert residency set size in inference performance in cases of limited memory. The expert capacity parameter C the relationship in *CacheFlow* that determines the number of experts which can reside in the GPU memory at any one given time. Various C can be used to study the trade-off between inference latency and expert eviction and memory-use.

Capacity	**Latency (s)**	**Evictions**	**Swap Volume (MB)**	**Hit Rate (%)**
4	7.619	2841	12802.5	10.2
8	4.594	2508	11322.0	20.6
12	6.667	2186	9891.0	30.6
16	8.540	1903	8635.5	39.4
32	8.579	1175	5431.5	61.9
60	7.793	0	270.0	98.1

Table 4.1: Performance Metrics at Different Capacities

At very low setting of C system often evicts and reloads experts. The correlation between expert capacity and inference delay under memory constraints is shown in Figure 4.1. In this case, it is the duty of the scheduler to always substitute resident experts to accommodate new execution requests. As a result interpretation delay increases and it becomes erratic because of too much switching of experts. This is a case of a cache thrashing, as the amount of work demanded by the experts is more than the capacity.

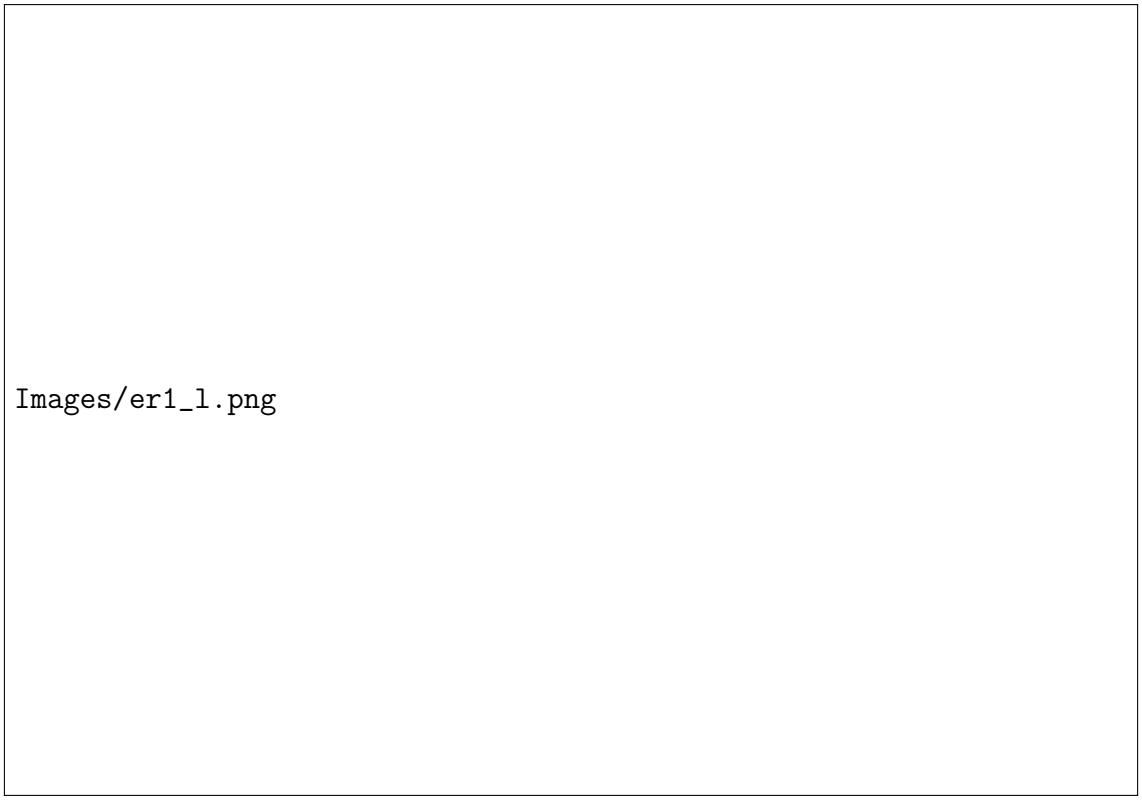


Figure 4.1: Latency vs. expert cache capacity.

The higher C, the more experts used by *CacheFlow* can retain. It has been experimentally demonstrated that the rate of eviction decreases sharply with C and inference latency decreases. Beyond some stage, there are smaller benefits to additional capacity, signifying that the active expert set is entirely represented.

This stability is seen about C=16 across the analyzed workloads. Figure 4.2 illustrates how eviction pressure decreases as capacity increases. And at such capacity, there is no inference latency variance between tokens and also, cache thrashing is substantially reduced. Notably, this amount of working-set is much less than the number of expert being in the model, which demonstrates that all experts need not be loaded to achieve stable performance.

These findings attest to the notion that MoE inference has a high level of locality in expert utilization, and the resident set can be relatively small before performing with high efficiency. The *CacheFlow* uses this property by setting the notion of expert residency dynamically according to observed demand, providing static inference as well as staying within hard-coded memory constraints. Determining a working-set capacity proves that management of expert level memory is a feasible and good mechanism of deploying large MoE models on limited hardware.

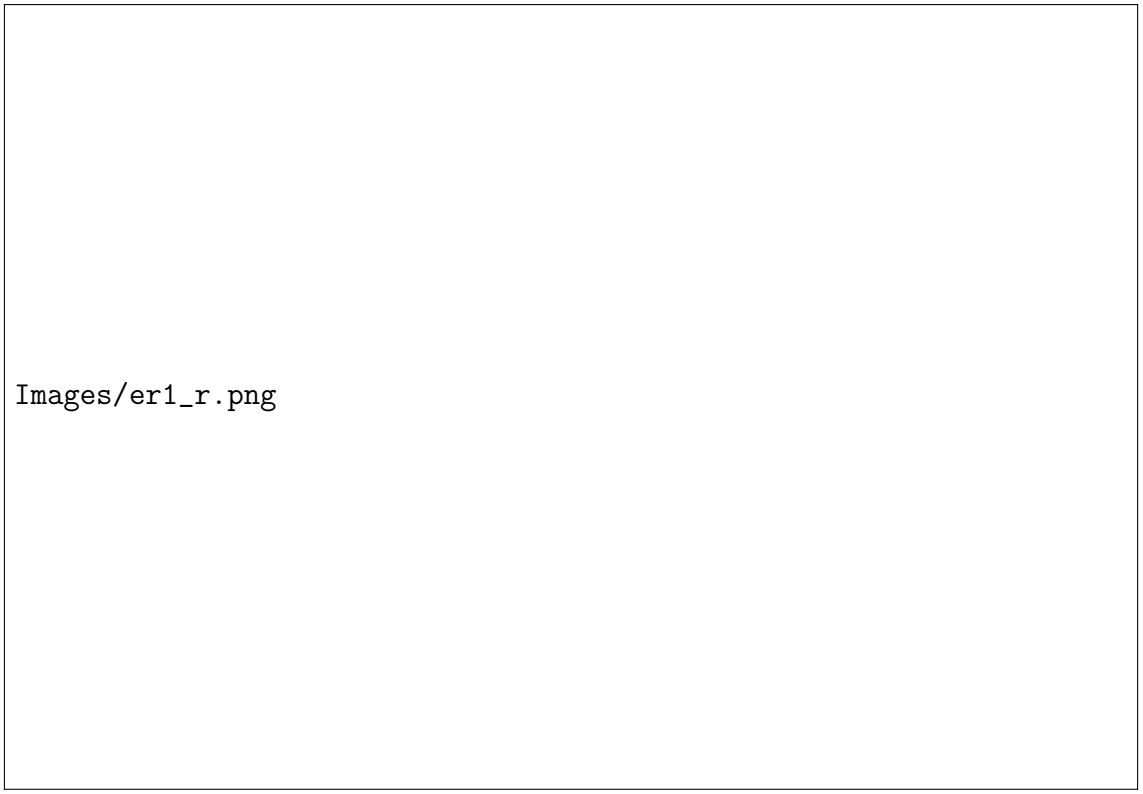


Figure 4.2: Evictions vs. expert cache capacity.

4.2 Behavioral Explainability

In addition to the aggregate performance measures, it is important to understand how the *CacheFlow* scheduler is internally behaving and how its decisions vary when it is in the process of inference. To that end, we conducted further analysis to study expert residency patterns and utilization distributions which provided the insight into the expert selection and cache management dynamics.

4.2.1 Expert Residency and Temporal locality

The analysis of expert residency through time shows that there is great temporal locality in expert usage. Some specialists are repeatedly selected in several tokens, and this is why they stay in the memory of the graphics cards over a long period. The same pattern can be observed in the residency heat maps, where a restricted number of specialists take the active set over as the rest of the ones are used infrequently.Figure 4.3 illustrates this temporal location in expert usage.

The locality of *CacheFlow* is enhanced by the priority of the mechanism which gives preference to already resident experts attached to existing key-value contexts. By implication, the experts that are utilized regularly will automatically be referred

Figure 4.3: Expert cache residency (C = 16).

to the top of the virtual LRU hierarchy and guaranteed against being pushed off. This is a self-stabilizing behavior that allows the scheduler to reach a stable working set that does not need to be explicitly tuned or even known about expert usage behaviors.

4.2.2 Expert Utilization and Fairness

On the other hand, *CacheFlow* emphasizes on locality, though it is also necessary to make sure that scheduler is not focused on one expert or a small set of experts. Expert usage distributions analysis indicates that though it is skewed, the execution is distributed among numerous experts. This proves that the gating network has been maintained with routing tokens to various experts and that the scheduler does not override routing decisions but abides by them. Figure 4.4 displays the pattern of expert use across inference requests.

Analyzing the patterns of utilization, it can be completed that *CacheFlow* maintains the functional behavior of the MoE model. The scheduling policy does not ground experts or artificially constrain them, but rather, the rate of their execution may be based on real routing demand. The task of the scheduler is to control the residency and order of execution, but to leave the routing behavior learnt by the model unchanged.

Figure 4.4: Expert utilization distribution.

4.2.3 Interpretability of Scheduling Decisions

It is also easy to adjust the data within a priority scoring, which also increases the meaningfulness of the behavior of *CacheFlow*. Every one of the scheduling decisions may be tracked to the physical indicators like reuse of context and status of expert residency. This scholarly clearness processes it workable to access reasonable thinking concerning framework conduct amid divergent workloads, and to trouble-troubleshoot or categorize the framework without imparting so-called black box intuitions.

In general, this behavioral analysis attests that *CacheFlow* is a predictable and explainable behavior. The framework uses temporal locality to stabilize expert residency, and ensures equity among experts, as well as carrying out the desired routing dynamics of the underlying MoE model. These properties are critical in the implementation of memory aware scheduling system of inference systems.

4.3 Impact of Prompt Locality

The *CacheFlow* performance is determined based on the type of access patterns developed due to the input workloads. Figure 4.5 shows a comparison of cache hit rates for various quick workloads.Mixture-of- Experts models use token content in routing, and thus semantically similar prompts tend to activate the exact same set of experts. In order to understand the effect of this to scheduling we experimented on the workloads of varying prompt locality.

There were two types of workloads, which included coherent-topic prompts and random-topic prompts. Coherent prompts involve successive questions posed within

a domain specializing e.g. physics questions. The patterns of prompts elicit replication of consistent patterns of expert activated events across multiple tokens. Random prompts, in contrast, are the mixture of dissimilar questions representing numerous topics and yield an extremely varied expert routing. Figure 4.5 shows a comparison of cache hit rates for various quick workloads.

Figure 4.5: Cache hit rate under different prompt locality.

The experiments demonstrate that experiments using join-coherent work loads result in considerably a higher rate of cache hits compared to those with random work loads. In case of a locality, the same experts are continually selected, and *CacheFlow* can store them in memory and eliminate avoidable evictions. This is multiplied by the priority-based scheduler with the preference it gives to resident experts and the context of key-value that results in smoother execution and reduced latency.

Random-topic workloads also exhibit less predictable access patterns of experts, thus, there are several requests made by non-resident experts. This increases the rate of eviction as well as reduces the efficiency of the cache since it reduces the capacitive

level. Nevertheless, under such a challenging situation, *CacheFlow* is right and does not crash due to out-of-memory and is considered to be robust in the case that routing conditions do not favor.

Those findings prove that the *CacheFlow* implicitly encourages the semantic locality without any explicit workload modeling or timely reordering. Through matching the expert access patterns with occurrences in expert residency, the framework also automatically succeeds in optimizing the value of expert reuse, which highlights the significant connection between the semantics of the model and schedule of the system.

4.4 Memory Optimization Analysis

The main goal of *CacheFlow* is to reduce the size of the memory footprint of Mixture-of-Experts inference without affecting either accuracy or performance. To measure the effectiveness of the framework, we have to differentiate between the lost knowledge in expert memory and the memory of the whole system, since each is a different measure of usage.

4.4.1 Reduction in Active Expert Memory

During a standard run of the MoE inference, all experts are loaded into a GPU memory possibly with majority of them being idle. Every 60 experts are in memory in our Qwen2-MoE tests. *CacheFlow* substitutes this fixed loading with a limited amount of expertise model that maintains a preset amount of experts in the GPU memory simultaneously.

Experiments indicate that when the expert capacity is set to C=16 to reduce the active set to the same number 16 experts: an approximate 73% reduction in expert memory is merely due to dynamic scheduling and eviction. This savings has no effect on the routing decisions or numeric correctness of the model, since still the gating network chooses the correct experts when the demand occurs.

4.4.2 Total GPU Memory Reduction

Expert weights prevail in the memory budget, but are not only contributors. The token embeddings, attention layers, normalizations parameters, shared projections, are all examples of such static components, that remains in place during the inference and not reduced down to experts.

Therefore, the total VRAM decrease is not as much compared to that of the expert-memory decrease. It is measured that CacheFlow can attain an overall 47-50% reduction in VRAM, based on workload. In practice, a model with 9 GB or so can execute in 4.5 GB using *CacheFlow*.

4.4.3 Practical Implications

The dichotomous existence of the expert memory and total memory is the crucial concept in interpreting the influence of *CacheFlow*. The framework demonstrates that instead of removing the entire overhead, it is possible to manage the essential part of the footprint (expert parameters) dynamically and retrench it significantly. The constant component is retained as a reference point that can be optimized upon in future (quantization, redesigning, etc.).

These memory savings allow large MoE models to execute on consumer GPUs having 6-8GB of memory, at which traditional procedures would be exhausted. With the help of memory, *CacheFlow* transforms MoE inference into a feasible solution on a small-scale hardware.

4.5 Discussion

Experimental results demonstrate that *CacheFlow* alters simultaneously the possibilities of Mixture-of-Experts (MoE) inference on hardware having a limited amount of memory. In all test configurations, even the standard, i.e. placing all experts on the GPU, the test did not succeed due to out-of-memory (OOM) errors. This is a demonstration the fact that of the conventional limits of MoE inference is not only inefficient, it is impossible under real VRAM constraints. In this case, *CacheFlow* is not an excellent performance optimization, it is a necessity to make the inference go.

The performance analysis shows that MoE inference reacts like an ordinary working set. Although the model has 60 experts, we were able to reach stable and low latency inference with 16 experts in memory. This demonstrates that the real need of experts in the process of inference is significantly smaller than the number of the sets. *CacheFlow* exploits this fact by entering and leaving experts into memory as it notices their use, thereby ensuring that the system runs well without having experts in the system.

It is also revealed through behavioral analysis that the scheduling provided by *CacheFlow* is being stable and intuitive. Expert residency is evidently locally in

time, and the usage histogram demonstrates that the computation is not centered on any individual expert, or even a small fixed set. The routing represented by the model is maintained by the scheduler and only interfered with to control memory and execution order. This distinction ensures that *CacheFlow* rejects model flexibility or model correctness.

The analysis of prompt locality sheds more light on the impact of workload structure on the performance of a system. Coherent topic prompts significantly increase the rate of cache hits and reduces the number of evictions, i.e. semantic structure of the application directly can be associated with the improvement of the system efficiency. Notably, *CacheFlow* can achieve these advantages and does not explicitly model the workload, or re-organize prompts; instead, it uses lightweight priority signals which are derived at runtime.

Regarding the issue of memory, the findings explain the constraint and advantage of dynamic management of experts. When the use of expert memory was cut by nearly 73% and total GPS memory consumption was also reduced to around 47 -50% by the need to have static elements always remain in memory. This result illustrates the importance of using expert scheduling in combination with alternative methods, e.g. quantization. And collectively, they enabled large MoE models, which were once available only on high-end hardware, to run on the consumer GPUs.

In short, the results demonstrate that memory conscious expert scheduling is an efficient and viable way of MoE inference. *CacheFlow* demonstrates how even thin models demonstrate the previously obscure performance of the sparse architectures: by integrating scheduling logic into the execution path of the model they can be transformed into real-world systems, using the large MoE models as the theoretical constructs they intended to be.

4.6 Summary

This chapter measured the performance, behaviour and memory utilization of *CacheFlow* running in actual inference workloads. The research revealed that the count of the professionals retained in the GPU memory is critical regarding both the stability and latency. Experiments that altered the expert capacity found that there is a clear working-set effect by which steady communication is attained when merely a small company of expert remains in GPU memory.

Analysis of behavior indicated that *CacheFlow* observes temporal locality in ex-

pertise application without using experts unequally. Expert residency and utilization patterns revealed that the scheduler will automatically switch to the demand routing to prevent too much work being concentrated in the hand of a few experts. And measuring prompt locality also demonstrated that semantically consistent workloads enhance cache performance, which results in the importance of workload aware execution in MoE systems.

Regarding memory, the study differentiated between the reduction of just expert set and overall reduction in the use of the GPU memory. With the help of static model components that remain fixed, *CacheFlow* reduced the active expert set by approximately 73 percent and reduced overall VRAM consumption by 47 - 50%. These cuts allow very large MoE models to execute on hardware which would otherwise exhaust its memory with more standard methods of inference.

Overall, the results show that *CacheFlow* is a practical and efficient way to handle MoE inference when memory is constrained. By including priority-based scheduling into the model execution flow, the framework enables big MoE models to run on consumer-grade hardware with performance guarantees. The thesis is concluded with a wrap-up chapter that summarizes the findings and offers recommendations for further research.

Chapter 5

Conclusion

5.1 Research Summary

The thesis discussed the issue of how to execute large Mixture-of Experts (MoE) language models on devices with small memory. As MoE systems reduce the computation costs by loading only a small number of experts because they are activated, common inference algorithms load all experts in the GPU memory, which is not consumer-friendly with large models. To address this, the paper proposed *CacheFlow* a priority dynamic scheduling model that provides the efficient inference of MoE without violating strict VRAM constraints. *CacheFlow* was developed as Python middleware and mounted directly on a forward pass of a manually recompiled Qwen2-MoE model. The framework perceives expert weights as a dynamic resource, as opposed to a fixed allocation, by decoupling scheduling and execution of models. The algorithm manages experts based on the degree of key-value context they are used and their expected duration in residence as well as on restricting the number of experts that may be resident at a given time. It was demonstrated through numerous experiments that one can arrive at stable inference using a small number of experts. Maintaining an amount of only 16 of 60 experienced removed thrashing of the cache and balanced out latency. Dynamic management reduced active expert memory by approximately 73% and overall GPU memory consumption by 47-50% permitting large models of MoE to map on hardware, otherwise crashing due to out-of-memory errors. In addition to performance and memory, the paper has examined the internal behavior of *CacheFlow*. Results indicated that it maintains the routing model of MoE, can retain experts that are fair, and it can benefit highly with locality of reference. Collectively, these findings demonstrate that memory conscious expert scheduling is a viable methodology that can be used to bridge the gap between the theoretical scalability of MoE models, and their practical implementation.

5.2 Contribution of the Work

The thesis contributes to the research on memory-efficient Mixture-of-Experts inference in a number of ways. It presents *CacheFlow*, an expert-scheduling system, first, in which the priorities are set by the introduction of a priority. *CacheFlow* can successfully execute large MoE models at bandwidth-intensive running durations when it is trying to use out of memory error (conventional inference usually fails). Second, the work demonstrates that the expert parameters may be addressed as an actively controlled resource. This is done by beyond just using the scheduling logic as part of the forward pass of the model by manually reconstructing the Qwen2-MoE architecture. Third, the experiment analysis offers empirical data about a small working set of experts. It proves that stable inferences can be made using just a small percentage of the total experts that are in memory. Lastly, the paper points out the effect of timeliness of locality on the effectiveness of the cache. It provides system level information about the semantics of workloads and expert scheduling behavior. These contributions combined with further work on consumer-grade hardware are positive steps toward enabling MoE models to be practically deployed and provision a basis of future research conducting yet in this area.

5.3 Future Work

Although *CacheFlow* has proven that priority-based expert scheduling is plausible and effective, there are still a number of ways in which the approach can be improved. Low level optimization is one significant extension. Moving the scheduler implementation out of Python code into C++ code will remove overhead incurred by interpreters, and alleviate the effects of the Global Interpreter Lock (GIL), and thus reduce scheduling latency and increase throughput. A second strategy is based on predictive prefetching. A look-ahead would predict the specialists needed in future tokens and preload them during processing of an already running token, which would also minimize the number of stalls due to expert admission. Lastly, the framework can be extended with the help of the extended architectural support. It would be interesting to demonstrate the generality of the approach to a range of different MoE designs by extending the *CacheFlow* wrapper to support new MoE models like Mixtral 8x7B and Grok-1. The extensions would additionally make CacheFlow a system that is scalable and production ready with memory limited MoE inference.

References

[1] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, 1991.

[2] M. I. Jordan and R. A. Jacobs, "Hierarchical mixtures of experts and the em algorithm," *Neural Computation*, 1994.

[3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[4] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[5] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *International Conference on Learning Representations (ICLR)*, 2015.

[6] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International Conference on Machine Learning (ICML)*, 2015.

[7] L. J. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[8] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," *arXiv preprint arXiv:1606.08415*, 2016.

[9] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[10] N. Shazeer, A. Mirhoseini, K. Maziarz, *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations (ICLR)*, 2017.

[11] E. Grave, A. Joulin, M. Cissé, D. Grangier, and H. Jégou, "Efficient softmax approximation for gpus," in *International Conference on Machine Learning (ICML)*, 2017.

[12] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," tech. rep., OpenAI, 2018.

[13] Z. Yang, Z. Dai, R. Salakhutdinov, and W. W. Cohen, "Breaking the softmax bottleneck: A high-rank rnn language model," in *International Conference on Learning Representations (ICLR)*, 2018.

[14] A. Radford, J. Wu, R. Child, *et al.*, "Language models are unsupervised multitask learners," tech. rep., OpenAI, 2019.

[15] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," in *arXiv preprint arXiv:1904.10509*, 2019.

[16] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[17] T. B. Brown, B. Mann, N. Ryder, *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[18] C. Raffel, N. Shazeer, A. Roberts, *et al.*, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research (JMLR)*, 2020.

[19] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2020.

[20] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2020.

[21] N. Shazeer, "Glu variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[22] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[23] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "Albert: A lite bert for self-supervised learning of language representations," in *International Conference on Learning Representations (ICLR)*, 2020.

[24] D. Lepikhin, H. Lee, Y. Xu, *et al.*, "Gshard: Scaling giant models with conditional computation and automatic sharding," in *International Conference on Learning Representations (ICLR)*, 2021.

[25] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research (JMLR)*, 2022.

[26] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[27] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[28] H. Touvron, T. Lavril, G. Izacard, *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[29] H. Yoo and H. Lee, "Swapmoe: Serving off-the-shelf moe-based large language models with tunable memory budget," *arXiv preprint arXiv:2309.09515*, 2023.

[30] S. Zuo *et al.*, "Moe-infinity: Efficient moe inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2305.11794*, 2023.

[31] Microsoft, "Deepspeed-mii: Model inference interface," 2023.

[32] NVIDIA, "Tensorrt-llm: Optimized inference for large language models," 2023.

[33] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[34] W. Liang *et al.*, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[35] X. Shen *et al.*, "fmoe: Fine-grained and prompt-aware expert offloading for large mixture-of-experts serving," *arXiv preprint arXiv:2403.12881*, 2024.